

Python Programming

A Practical Introduction

Hoa T. Vu

Variables

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10          # int
pi = 3.14       # float
name = "Alice"  # str
flag = True     # bool
```

```
print(x)        # 10
print(type(x))  # <class 'int'>

x = x + 1       # reassign
x += 1          # shorthand
```

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10                # int                print(x)                # 10
pi = 3.14             # float              print(type(x))           # <class 'int'>
name = "Alice"        # str
flag = True           # bool

x = x + 1             # reassign
x += 1                # shorthand
```

- No declaration needed, just assign
- Types can change: `x = 10` then `x = "hello"` is valid

Common Operations

Arithmetic

```
7 + 2    # 9
7 - 2    # 5
7 * 2    # 14
7 / 2    # 3.5    (float)
7 // 2   # 3      (floor div)
7 % 2    # 1      (remainder)
7 ** 2   # 49     (power)
```

Strings

```
s = "hello"
len(s)                # 5
s.upper()             # "HELLO"
s[0]                  # "h"
s[1:4]                # "ell"

# f-string
n = 42
f"answer: {n}"        # "answer: 42"
```

Lists

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]           # 3           (first element)
```

```
nums[-1]          # 5           (last element)
```

```
nums[1:3]         # [1, 4]     (slice)
```

```
len(nums)         # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]          # 3          (first element)
```

```
nums[-1]         # 5          (last element)
```

```
nums[1:3]        # [1, 4]    (slice)
```

```
len(nums)        # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists can hold mixed types: [1, "hello", 3.14, True]

List Comprehension

Build a list in one line more readable than a loop.

```
# squares of 0..4  
squares = [x**2 for x in range(5)]  
# [0, 1, 4, 9, 16]
```

```
# even numbers only  
evens = [x for x in range(10) if x % 2 == 0]  
# [0, 2, 4, 6, 8]
```

```
# nested: 2-D grid of (row, col) pairs  
pairs = [(r, c) for r in range(3) for c in range(3)]
```

Dictionaries

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student          # True  
student.get("phone", "N/A") # "N/A" if missing
```

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student          # True  
student.get("phone", "N/A") # "N/A" if missing
```

Keys must be *immutable* (strings, numbers, tuples). Values can be anything.

Iterating Dictionaries

```
grades = {"Alice": 90, "Bob": 85, "Carol": 92}
```

```
# keys
```

```
for name in grades:  
    print(name)
```

```
# values
```

```
for score in grades.values():  
    print(score)
```

```
# both
```

```
for name, score in grades.items():  
    print(f"{name}: {score}")
```

Conditionals

if / elif / else

Python uses **indentation** (4 spaces) to define blocks; no braces.

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C"
```

Comparison Operators

- `==` equal `!=` not equal
- `<` `<=` `>` `>=` ordering
- `and` `or` `not` logical
- `in` membership test

```
x = 5
x > 3 and x < 10    # True
x in [1, 3, 5, 7]   # True
not (x == 4)        # True
```


Loops

for Loop

iterate over a list

```
for x in [2, 4, 6]:  
    print(x)
```

iterate with index

```
for i, x in enumerate([2, 4, 6]):  
    print(i, x)
```

range(start, stop, step)

```
for i in range(5):                # 0, 1, 2, 3, 4  
    print(i)
```

```
for i in range(0, 10, 2):        # 0, 2, 4, 6, 8  
    print(i)
```

while Loop

```
n = 1
while n < 100:
    n *= 2
print(n)    # 128
```

break and continue

```
for x in range(10):
    if x == 5:
        break           # exit the loop entirely
    if x % 2 == 0:
        continue        # skip to next iteration
    print(x)             # prints 1 3
```

- Python is widely used for data science and machine learning development.

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)
- You can make Python code faster using Cython.

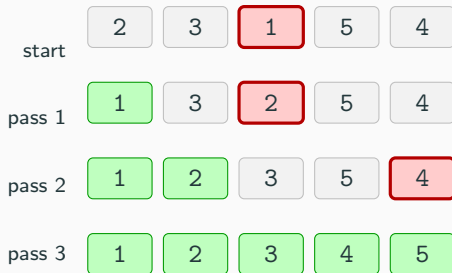
Introduction to Algorithms

- Sequence of instructions that solve a problem
- Running time: number of computer instructions (memory access and pairwise arithmetics)

Selection Sort

Main idea:

- Find the **minimum** element in the unsorted portion
- **Swap** it into the first unsorted position
- Repeat, shrinking the unsorted portion by one each time



Selection Sort — Code

```
def selection_sort(a):  
    n = len(a)  
    for i in range(n):  
        next_min = i  
        for j in range(i + 1, n):  
            if a[j] < a[next_min]:  
                next_min = j  
        a[i], a[next_min] = a[next_min], a[i]  
  
a = [5, 3, 8, 1, 4]  
selection_sort(a)  
# [1, 3, 4, 5, 8]
```

Selection Sort — Code

```
def selection_sort(a):  
    n = len(a)                                #1  
    for i in range(n):                        #2  
        next_min = i                          #3  
        for j in range(i + 1, n):            #4  
            if a[j] < a[next_min]:             #5  
                next_min = j                  #6  
    a[i], a[next_min] = a[next_min], a[i]     #7
```

Selection Sort — Running Time

We count the number of times each line runs:

- Lines #1, #3, #7: run n times (once per outer iteration)
- Lines #4–#6: inner loop runs

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2}$$

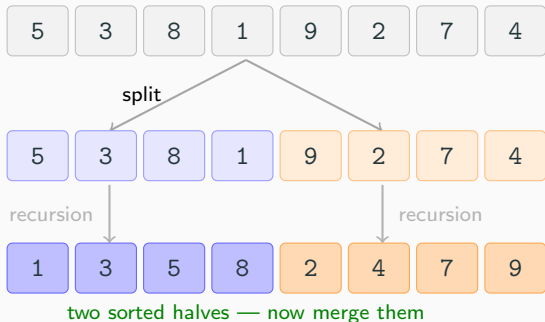
times in total

- All other lines: constant or $O(n)$ — negligible

Total $\approx \frac{n^2}{2}$ operations $\Rightarrow$ **running time** = $O(n^2)$.

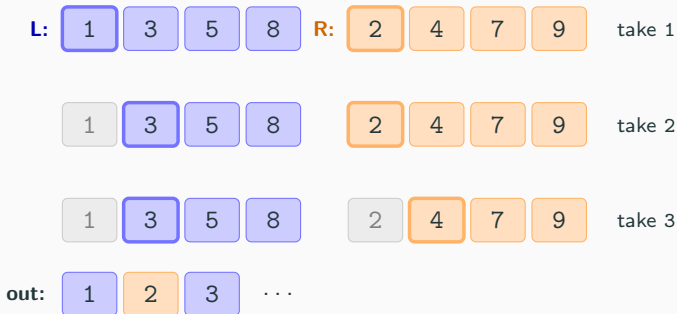
Merge Sort — Recursion

- **Split** the array into two halves
- **Recursively** sort each half (trust the recursion!)
- **Merge** the two sorted halves (next slide)



Merge Sort — Merge Step

Compare front elements of L and R; take the smaller one. Repeat.



Merge Sort — Code (merge)

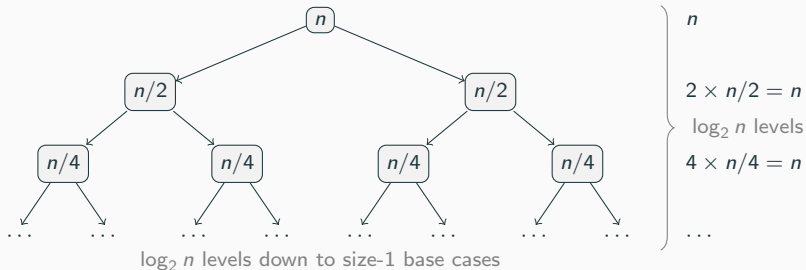
```
def merge(left, right):  
    out = []  
    i = j = 0  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            out.append(left[i]); i += 1  
        else:  
            out.append(right[j]); j += 1  
    out += left[i:]    # append remaining elements  
    out += right[j:]  
    return out
```

Merge Sort — Code (merge_sort)

```
def merge_sort(a):  
    if len(a) <= 1:          # base case  
        return a  
    mid = len(a) // 2  
    left  = merge_sort(a[:mid])  
    right = merge_sort(a[mid:])  
    return merge(left, right)
```


Merge Sort — Running Time

Each level of the recursion tree does $O(n)$ total work; there are $\log_2 n$ levels.



Total work = $n \times \log_2 n \Rightarrow$ **running time** = $O(n \log n)$.

NumPy

NumPy Arrays

NumPy provides fast, vectorized operations on arrays.

```
import numpy as np

a = np.array([1, 2, 3, 4])    # 1-D array
A = np.array([[1, 2],
               [3, 4]])       # 2-D array (matrix)

a.shape    # (4,)
A.shape    # (2, 2)
A.dtype    # float64 / int64 ...

np.zeros((3, 3))    # 3x3 zero matrix
np.ones((2, 4))     # 2x4 ones
np.eye(3)           # 3x3 identity
np.random.randn(3, 3) # random normal
```

Indexing and Slicing

```
A = np.array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
A[0, 1]      # 2           (row 0, col 1)  
A[1, :]      # [4, 5, 6]  (entire row 1)  
A[:, 2]      # [3, 6, 9]  (entire col 2)  
A[:2, :2]    # [[1,2],[4,5]] (top-left 2x2)
```

```
# boolean mask
```

```
a = np.array([1, -2, 3, -4])  
a[a > 0]      # [1, 3]  
a[a > 0] = 0  # set positives to zero
```

Vectorized Operations

Operations apply **element-wise** no explicit loops needed.

```
a = np.array([1.0, 2.0, 3.0])
```

```
b = np.array([4.0, 5.0, 6.0])
```

```
a + b          # [5., 7., 9.]
```

```
a * b          # [4., 10., 18.]
```

```
a ** 2         # [1., 4., 9.]
```

```
np.sqrt(a)     # [1., 1.41, 1.73]
```

```
np.exp(a)      # [e, e^2, e^3]
```

```
# scalar broadcast
```

```
a * 3          # [3., 6., 9.]
```

```
a + 10         # [11., 12., 13.]
```

Linear Algebra

```
A = np.array([[1., 2.], [3., 4.]])
b = np.array([5., 6.])

# dot product / matrix-vector product
np.dot(a, b)          # scalar
A @ b                 # matrix-vector: shape (2,)
A @ A                 # matrix-matrix product

# transpose
A.T                   # [[1,3],[2,4]]

# norms
np.linalg.norm(b)     # L2 norm
np.linalg.norm(b, ord=1) # L1 norm

# solve  $Ax = b$ 
x = np.linalg.solve(A, b)
```

Reductions and Statistics

```
A = np.array([[1., 2., 3.],  
              [4., 5., 6.]])
```

```
A.sum()           # 21.0   (all elements)
```

```
A.sum(axis=0)     # [5., 7., 9.] (column sums)
```

```
A.sum(axis=1)     # [6., 15.]  (row sums)
```

```
A.mean()          # 3.5
```

```
A.max()           # 6.0
```

```
A.argmax()        # 5   (flat index)
```

```
A.argmax(axis=1)  # [2, 2] (col index of max per row)
```

Reductions and Statistics

```
A = np.array([[1., 2., 3.],  
              [4., 5., 6.]])
```

```
A.sum()           # 21.0   (all elements)
```

```
A.sum(axis=0)     # [5., 7., 9.] (column sums)
```

```
A.sum(axis=1)     # [6., 15.]  (row sums)
```

```
A.mean()          # 3.5
```

```
A.max()           # 6.0
```

```
A.argmax()        # 5   (flat index)
```

```
A.argmax(axis=1)  # [2, 2] (col index of max per row)
```

Rule of thumb: if you are writing a Python `for` loop over array elements, there is usually a NumPy function that does it faster.

Dynamic Programming Fibonacci

Naive recursion $O(2^n)$

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

fib(5) recomputes fib(3) twice,
fib(2) three times, ...

DP (bottom-up) $O(n)$

```
def fib_dp(n):  
    dp = [0] * (n + 1)  
    dp[1] = 1  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
    return dp[n]
```

Each subproblem solved **once**, result stored.

Subset Sum Idea

Problem: given integers `nums` and target T , does any subset sum to T ?

State: `dp[i][s]` = True if some subset of the first i numbers sums to s .

Recurrence:

$$\text{dp}[i][s] = \underbrace{\text{dp}[i-1][s]}_{\text{skip item } i} \vee \underbrace{\text{dp}[i-1][s - \text{nums}[i-1]]}_{\text{take item } i}$$

Base case: `dp[0][0]` = True (empty subset sums to 0).

Subset Sum Code

```
def subset_sum(nums, T):  
    n = len(nums)  
    dp = [[False] * (T + 1) for _ in range(n + 1)]  
    dp[0][0] = True  
    for i in range(1, n + 1):  
        for s in range(T + 1):  
            dp[i][s] = dp[i-1][s]  
            if s >= nums[i-1]:  
                dp[i][s] |= dp[i-1][s - nums[i-1]]  
    return dp[n][T]
```

Time: $O(nT)$ Space: $O(nT)$

Longest Increasing Subsequence (LIS)

Problem: find the length of the longest strictly increasing subsequence.

`arr = [3, 1, 4, 1, 5, 9, 2, 6]`

One LIS: $1 \rightarrow 4 \rightarrow 5 \rightarrow 6$ (length 4)

DP state: $dp[i]$ = length of LIS *ending at index i*

$$dp[i] = 1 + \max\{ dp[j] : j < i, arr[j] < arr[i] \}$$

LIS Code

```
def lis(arr):  
    n = len(arr)  
    dp = [1] * n          # every element is an LIS of length 1  
    for i in range(1, n):  
        for j in range(i):  
            if arr[j] < arr[i]:  
                dp[i] = max(dp[i], dp[j] + 1)  
    return max(dp)
```

Time: $O(n^2)$ Space: $O(n)$

Bonus: achievable in $O(n \log n)$ using binary search on a patience-sort stack.

Problems

Problem 1 Lists

Given `scores = [88, 72, 95, 61, 84, 90]`, write code to:

1. Access the third score.
2. Slice the last three scores.
3. Append 77, then sort in descending order.
4. Remove all scores below 80 using a list comprehension.

Problem 2 Loops

1. Write a loop that prints the first 10 Fibonacci numbers:
1, 1, 2, 3, 5, 8, 13, 21, 34, 55
2. Modify it to find the **first** Fibonacci number greater than 1 000.
3. Write a loop that finds all prime numbers up to 50.
Hint: a number n is prime if no integer from 2 to $\sqrt{n}$ divides it.

Problem 3 NumPy Linear Algebra

```
import numpy as np
A = np.array([[2., 1.], [5., 3.]])
b = np.array([4., 7.])
```

1. Solve $Ax = b$ with `np.linalg.solve`. Verify by checking $Ax \approx b$.
2. Compute the L1 and L2 norms of x .
3. Compute the eigenvalues of A .
4. What is `np.linalg.cond(A)`? What does a large condition number mean for numerical solving?

Problem 6a (hard) Subset Sum

Given $\text{nums} = [3, 1, 4, 7, 2]$, determine by hand which of the following targets can be formed as a subset sum, and exhibit a witness subset for each.

Target T	Achievable?	Subset
8	?	?
10	?	?
6	?	?
15	?	?

Problem 6b (hard) Subset Sum

Implement the DP solution from the slides for `subset_sum(nums, T)`.

1. Verify your implementation on the four targets from part (a).
2. Trace through the dp table by hand for `nums = [2, 3, 5]`, `T = 5`. Show the full table.
3. What is the time and space complexity?
4. (*Bonus*) Modify the function to return the actual subset, not just True/False.
Hint: backtrack through the table.

Problem 7 (hard) Counting Inversions

An **inversion** in a list is a pair (i, j) with $i < j$ but $\text{arr}[i] > \text{arr}[j]$.

```
arr = [3, 1, 2, 5, 4]
```

```
# inversions: (3,1), (3,2), (5,4) -> count = 3
```

1. Write a brute-force solution using two nested loops. What is its time complexity?
2. (*Bonus*) Write an $O(n \log n)$ solution.

Hint: think about merge sort. When merging two sorted halves, every element in the right half that is placed before an element in the left half contributes exactly (number of remaining elements in the left half) inversions.